

Explorando el Ecosistema de Python para Inteligencia Artificial

Dinámica práctica sobre TensorFlow

Investigación de una librería de Python utilizada en Ciencia de Datos e Inteligencia Artificial: qué es, para qué sirve, en qué situaciones conviene usarla y un ejemplo práctico ejecutado en local, incluyendo el ajuste previo del entorno Python y la verificación de compatibilidad con CUDA.

TensorFlow 2.21

Python 3.12

tf.keras

MNIST

CUDA / cuDNN

Redes neuronales

AUTOR

Miguel Jericó

FECHA

Julio 2026

Enunciado de la actividad

Objetivo: investigar una librería de Python utilizada en Ciencia de Datos o Inteligencia Artificial y explicar al resto de la clase para qué sirve y en qué situaciones se utiliza. La actividad combina exposición conceptual del framework elegido con una demostración práctica ejecutada en el equipo local del alumno, de modo que se verifique no solo el conocimiento teórico sino también la capacidad de preparar el entorno, instalar dependencias y ejecutar código real.

Librería investigada

TensorFlow — plataforma de código abierto creada por Google para construir, entrenar y desplegar modelos de aprendizaje automático e inteligencia artificial.

1 Preparación del entorno: por qué hacemos downgrade a Python 3.12

Antes de instalar TensorFlow es necesario ajustar la versión de Python. En mi equipo tenía instalado Python 3.14, que es una versión extremadamente reciente y todavía no es compatible con TensorFlow. Por ello he hecho un *downgrade* a Python 3.12 por tres motivos técnicos, todos ellos documentados en las notas de versión oficiales del proyecto TensorFlow y del ecosistema CUDA de NVIDIA.

1.1 Desfase en el ciclo de lanzamientos

Python 3.14 es una versión extremadamente reciente. El equipo de desarrollo de Google y los contribuidores de TensorFlow diseñan cada actualización basándose en versiones de Python estables y maduras. Actualmente, la versión estable más reciente de TensorFlow (TensorFlow 2.21) tiene soporte optimizado únicamente para Python 3.10 hasta Python 3.13. Aún no se han compilado los archivos binarios oficiales (*wheels*) para la estructura interna de Python 3.14, por lo que cualquier intentar instalar TensorFlow sobre esa versión produce un error inmediato.

1.2 Dependencias de C/C++ y controladores de NVIDIA (CUDA)

A diferencia de las librerías comunes de Python, que son puro código de texto, TensorFlow es en realidad un motor gigante escrito en C++ que se comunica con el hardware. Para usar la GPU, TensorFlow necesita enlazarse con los controladores de NVIDIA (CUDA Toolkit y cuDNN). Estas herramientas de NVIDIA deben ser testeadas y adaptadas para trabajar en conjunto con los cambios internos que introduce cada nueva versión de Python, especialmente en la API de C de Python. Al ser Python 3.14 tan nuevo, NVIDIA y Google aún no han certificado esa combinación exacta.

1.3 La ausencia de «wheels» precompilados

Cuando se ejecuta `pip install tensorflow`, el gestor busca un paquete «pre-cocinado» (un archivo *.whl*) que coincida exactamente con el sistema operativo, la arquitectura (64 bits) y la versión exacta de

Python. Al no existir en los servidores de Python (PyPI) un paquete que diga expresamente «compatible con Python 3.14 y CUDA», pip se rinde de inmediato y lanza el error *No matching distribution found*.

Comando 1 — Instalación de Python 3.12 mediante winget

CMD · Windows Package Manager

```
winget install --id Python.Python.3.12 --override "/passive PrependPath=1"
```

```
C:\Windows\System32>winget install --id Python.Python.3.12 --override "/passive PrependPath=1"
Encontrado Python 3.12 [Python.Python.3.12] Versión 3.12.10
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://www.python.org/ftp/python/3.12.10/python-3.12.10-amd64.exe
25.7 MB / 25.7 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente
```

Captura 1 — Instalación de Python 3.12 en el equipo local.

Comando 2 — Instalación de TensorFlow con la versión específica de Python

CMD · py launcher

```
py -3.12 -m pip install tensorflow
```

```
C:\Windows\System32>py -3.12 -m pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp312-cp312-win_amd64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl.metadata (5.3 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Collecting packaging (from tensorflow)
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting protobuf<8.0.0,>=6.31.1 (from tensorflow)
  Downloading protobuf-7.35.1-cp310-abi3-win_amd64.whl.metadata (595 bytes)
Collecting requests<3,>=2.21.0 (from tensorflow)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting setuptools (from tensorflow)
  Downloading setuptools-82.0.1-py3-none-any.whl.metadata (6.5 kB)
Collecting six>=1.12.0 (from tensorflow)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting termcolor>=1.1.0 (from tensorflow)
  Downloading termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Collecting typing_extensions>=3.6.6 (from tensorflow)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting wrapt>=1.11.0 (from tensorflow)
  Downloading wrapt-2.2.2-cp312-cp312-win_amd64.whl.metadata (7.6 kB)
Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
  Downloading grpcio-1.81.1-cp312-cp312-win_amd64.whl.metadata (3.8 kB)
Collecting keras>=3.12.0 (from tensorflow)
  Downloading keras-3.15.0-py3-none-any.whl.metadata (6.3 kB)
Collecting numpy>=1.26.0 (from tensorflow)
  Downloading numpy-2.5.0-cp312-cp312-win_amd64.whl.metadata (6.6 kB)
Collecting h5py<3.15.0,>=3.11.0 (from tensorflow)
```

Captura 2 — Descarga e instalación del wheel de TensorFlow 2.21 para Python 3.12.

```

Administrador: Símbolo del sistema - py -3.12 -m pip install tensorflow
Downloading absl-py-2.4.0-py3-none-any.whl (135 kB)
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading gast-0.7.0-py3-none-any.whl (22 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading grpcio-1.81.1-cp312-cp312-win_amd64.whl (4.9 MB)
----- 4.9/4.9 MB 59.9 MB/s eta 0:00:00
Downloading h5py-3.14.0-cp312-cp312-win_amd64.whl (2.9 MB)
----- 2.9/2.9 MB 55.3 MB/s eta 0:00:00
Downloading keras-3.15.0-py3-none-any.whl (1.7 MB)
----- 1.7/1.7 MB 46.3 MB/s eta 0:00:00
Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl (26.4 MB)
----- 26.4/26.4 MB 57.7 MB/s eta 0:00:00
Downloading ml_dtypes-0.5.4-cp312-cp312-win_amd64.whl (212 kB)
Downloading numpy-2.5.0-cp312-cp312-win_amd64.whl (12.4 MB)
----- 12.4/12.4 MB 45.7 MB/s eta 0:00:00
Downloading opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading protobuf-7.35.1-cp310-ab31-win_amd64.whl (439 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Downloading wrapt-2.2.2-cp312-cp312-win_amd64.whl (80 kB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
Downloading setuptools-82.0.1-py3-none-any.whl (1.0 MB)
----- 1.0/1.0 MB 24.0 MB/s eta 0:00:00
Downloading certifi-2026.6.17-py3-none-any.whl (133 kB)
Downloading charset_normalizer-3.4.7-cp312-cp312-win_amd64.whl (159 kB)
Downloading idna-3.18-py3-none-any.whl (65 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading wheel-0.47.0-py3-none-any.whl (32 kB)
Downloading namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading optree-0.19.1-cp312-cp312-win_amd64.whl (341 kB)
Downloading rich-15.0.0-py3-none-any.whl (310 kB)
Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
----- 1.2/1.2 MB 31.3 MB/s eta 0:00:00
Installing collected packages: namex, libclang, flatbuffers, wrapt, urllib3, typing_extensions, termcolor, six, setuptools, pygments, protobuf, packaging, opt_einsum, numpy, mdurl, idna, gast, charset_normalizer, certifi, absl-py, wheel, requests, optree, ml_dtypes, markdown-it-py, h5py, grpcio, google_pasta, rich, astunparse, keras, tensorflow

```

Captura 3 — Salida final: TensorFlow instalado correctamente junto con sus dependencias (keras, numpy, protobuf, etc.).

Comando 3 — Actualización de pip

CMD · `pip upgrade`

```
py -3.12 -m pip install --upgrade pip
```

Con este comando podemos volver a actualizar el instalador de librerías, que también se había desactualizado con el cambio de versión de Python. Mantener pip al día es recomendable porque muchas dependencias de TensorFlow publican ruedas con metadatos que solo pip moderno sabe interpretar.

```

C:\Windows\System32>py -3.12 -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\paola\appdata\local\programs\python\python312\lib\site-packages (25.0.1)
Collecting pip
  Using cached pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
Using cached pip-26.1.2-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.0.1
    Uninstalling pip-25.0.1:
      Successfully uninstalled pip-25.0.1
Successfully installed pip-26.1.2

```

Captura 4 — pip actualizado a su versión más reciente para Python 3.12.

2 TensorFlow: qué es y para qué sirve

TensorFlow es una librería y plataforma de código abierto creada por Google para construir, entrenar y desplegar modelos de aprendizaje automático e inteligencia artificial. Se usa mucho en ciencia de datos porque permite trabajar con datos, redes neuronales y modelos predictivos de forma eficiente, tanto en ordenador como en web, móvil o nube. Su nombre proviene de los *tensores*, las estructuras de datos multidimensionales que constituyen la unidad básica de información con la que trabaja el motor interno de cálculo.

Definición formal (documentación oficial)

«TensorFlow es una plataforma integral de código abierto de extremo a extremo para aprendizaje automático. Cuenta con un ecosistema amplio y flexible de herramientas, librerías y recursos de la comunidad que permite a los investigadores avanzar en el estado del arte del ML y a los desarrolladores construir y desplegar fácilmente aplicaciones con ML.»

2.1 Historia y origen

TensorFlow tiene su origen en un proyecto interno de Google llamado **DistBelief**, desarrollado en 2011 por el equipo de Google Brain para entrenar redes neuronales profundas en clusters distribuidos. En noviembre de 2015, Google liberó una reescritura completa de DistBelief bajo el nombre de TensorFlow, con licencia Apache 2.0. La versión 1.x se basaba en un modelo de ejecución diferida (*graph mode*) en el que el usuario definía primero un grafo de operaciones y luego lo ejecutaba en una sesión.

En 2019 se publicó **TensorFlow 2.0**, que introdujo *eager execution* (ejecución inmediata, similar a como Python evalúa las operaciones) e integró Keras como API de alto nivel por defecto. Esto simplificó enormemente el desarrollo de modelos y acercó la plataforma a usuarios no expertos. Desde entonces, las sucesivas versiones (2.x hasta la 2.21 actual) han añadido mejoras de rendimiento, soporte para nuevas arquitecturas (TPU, aceleradores móviles) y herramientas como *tf.data*, *tf.function* y *TensorBoard*.

2.2 Arquitectura interna

Aunque a nivel de Python TensorFlow se presenta como una librería más, internamente es un sistema distribuido de cálculo numérico con varios niveles:

- **Front-end Python / C++:** la API que el usuario importa con *import tensorflow as tf*.
- **tf.keras:** API de alto nivel para definir modelos secuenciales o funcionales con pocas líneas de código.
- **Core runtime:** motor en C++ que compila el grafo de operaciones y lo ejecuta sobre CPU, GPU o TPU.
- **Eigen / cuBLAS / cuDNN:** librerías numéricas de bajo nivel optimizadas para cada tipo de hardware.
- **Distributed runtime:** permite entrenar un mismo modelo en varios dispositivos o nodos coordinados.

2.3 Para qué se utiliza

TensorFlow se utiliza cuando necesitamos crear modelos que aprendan a partir de datos. Es especialmente útil en tareas como clasificación de imágenes, reconocimiento de voz, procesamiento del lenguaje natural, predicción de valores y detección de patrones en grandes conjuntos de datos. También sirve para llevar esos modelos a producción, es decir, para que funcionen en aplicaciones reales una vez entrenados. Según la documentación oficial, TensorFlow permite crear modelos para escritorio, móviles (TensorFlow Lite), la web (TensorFlow.js) y la nube (TensorFlow Extended / TFX).

2.4 En qué situaciones conviene usarlo

Conviene usar TensorFlow cuando:

- Vas a trabajar con redes neuronales o aprendizaje profundo (*deep learning*).
- Necesitas escalar modelos y desplegarlos en distintos entornos (servidor, edge, navegador).
- Quieres hacer proyectos de visión artificial, NLP o predicción con datos estructurados.

- Buscas una herramienta consolidada con muchas APIs y recursos de apoyo (comunidad, tutoriales, modelos preentrenados).

Un ejemplo típico es entrenar una red neuronal para reconocer imágenes de dígitos escritos a mano, como el dataset **MNIST**, algo muy común en ejemplos introductorios de TensorFlow y que es precisamente el que utilizaremos en este trabajo.

2.5 Comparativa breve con PyTorch

Aunque TensorFlow es la opción más extendida en entornos productivos, conviene conocer su alternativa principal, PyTorch, desarrollada por Meta. La siguiente tabla recoge las diferencias más relevantes para un analista de datos:

Aspecto	TensorFlow	PyTorch
Creador	Google (2015)	Meta / Facebook (2016)
API de alto nivel	tf.keras (integrada)	torch.nn + Lightning
Modo de ejecución	Grafo + eager (tf.function)	Eager por defecto, más pythonic
Despliegue en producción	TF Serving, TFX, TF Lite, TF.js	TorchServe, ONNX export
Uso predominante	Industria, producción, edge	Investigación académica
Curva de aprendizaje	Moderada, más uniforme	Más suave para usuarios de Python

Tabla 1 — Comparativa sintética entre TensorFlow y PyTorch.

2.6 Qué aporta al ecosistema Python

En Python, TensorFlow destaca porque permite definir modelos de forma flexible con `tf.keras`, cargar datos, entrenar, evaluar y hacer inferencias en una misma biblioteca. También incluye herramientas para preparar datos (`tf.data`), visualizar métricas (*TensorBoard*) y optimizar el flujo de trabajo de IA mediante `tf.function`, que compila funciones Python a grafos de alto rendimiento. Un ejemplo sencillo de uso sería cargar un conjunto de imágenes, normalizar los datos, entrenar una red neuronal y luego evaluar su precisión sobre datos nuevos. Ese flujo es uno de los más habituales en proyectos de IA con Python.

2.7 Casos de uso reales

TensorFlow no es solo una herramienta académica: está detrás de productos que millones de personas utilizan a diario. Algunos ejemplos representativos son:

- **Google Photos:** clasificación automática de imágenes y búsqueda por contenido («perros», «playas», «rostros»).
- **Gmail / Google Workspace:** filtrado de spam y categorización inteligente del correo entrante.
- **Google Translate:** modelos neuronales de traducción (GNMT) que sustituyeron a los antiguos sistemas estadísticos.

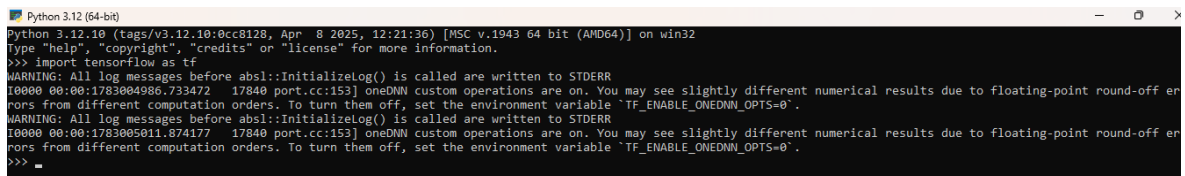
- **Airbnb:** clasificación de imágenes de los anuncios para detectar categorías de propiedad y mejorar la búsqueda.
- **Coca-Cola:** lectura automática de códigos de botellas mediante visión artificial en sus líneas de envasado.
- **Tesla:** componentes de su pila de conducción asistida utilizan modelos entrenados con frameworks de la familia TensorFlow.

3 Ejemplo práctico

Para empezar a utilizar TensorFlow en Python, abrimos la consola interactiva de Python y utilizamos el siguiente comando de importación:

Python · `import`

```
import tensorflow as tf
```



Captura 5 — Importación de TensorFlow sin errores: el entorno queda listo para usar la librería.

3.1 Estructura de la demostración

Vamos a implantar un pequeño script de **demostración básica en tres partes**. TensorFlow permite definir modelos con `tf.keras` y evaluar datos, por lo que el script muestra en primer lugar dos comprobaciones previas y, a continuación, el entrenamiento real de una red neuronal.

- **Parte 1:** demuestra qué es un *tensor* (el núcleo de la librería).
- **Parte 2:** descarga el dataset MNIST para probar que tu entorno está 100%% operativo.
- **Parte 3:** entrena una red neuronal simple durante una época y evalúa su precisión.

3.2 Script — Partes 1 y 2: tensores y carga de MNIST

Python · `demo_tf_partes_1_y_2.py`

```
# --- PARTE 1: COMPROBACIÓN Y TENSORES BÁSICOS ---
print("=== Iniciando Demostración de TensorFlow ===")
print(f"Versión instalada: {tf.__version__}")

# Crear dos tensores constantes (estructuras de datos de TensorFlow)
# Piensa en ellos como variables optimizadas para cálculos masivos
nodo1 = tf.constant(5.0)
nodo2 = tf.constant(6.0)

# Realizar una operación matemática usando el motor de TensorFlow
```



```

resultado = tf.multiply(nodo1, nodo2)

# Usamos .numpy() para extraer el valor en un formato legible estándar
print(f"\nMultiplicación de Tensores: {nodo1.numpy()} x {nodo2.numpy()} = {resultado.numpy()}")

# --- PARTE 2: CARGA DEL DATASET MNIST (Ejemplo del trabajo) ---
print("\n=== Descargando Dataset MNIST ===")
try:
    # Cargar el dataset desde la API de Keras integrada en TensorFlow
    mnist = tf.keras.datasets.mnist
    (x_train, y_train), (x_test, y_test) = mnist.load_data()

    # Normalizar los datos (convertir los valores de píxeles de 0-255 a 0.0-1.0)
    x_train, x_test = x_train / 255.0, x_test / 255.0

    print("¡Éxito! Dataset MNIST cargado y normalizado correctamente.")
    print(f"Cantidad de imágenes para entrenar: {len(x_train)}")
except Exception as e:
    print(f"Hubo un error al cargar Keras: {e}")

```

Nota técnica: la normalización $x_train / 255.0$ convierte los píxeles de escala 0–255 a 0.0–1.0. Esta escala mejora la estabilidad numérica del descenso de gradiente y acelera la convergencia del entrenamiento.

3.3 Script — Parte 3: creación y entrenamiento de la red neuronal

A continuación, y como tercera y última parte, utilizamos otro script para realizar un entrenamiento real, aunque sea fugaz; pero que nos permite, al fin y al cabo, ver cómo la máquina «aprende» en tiempo real. Vamos a configurar una red neuronal muy sencilla y la entrenaremos con una sola «época» (una única pasada por los datos). Tardará apenas unos segundos. Este es el bloque completo de código que usaríamos:

Python · [demo_tf_parte_3.py](#)

```

# --- PARTE 3: CREACIÓN Y ENTRENAMIENTO DE LA RED NEURONAL ---
print("\n=== Construyendo la Red Neuronal ===")

# 1. Definir la arquitectura (modelo secuencial básico)
modelo = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)), # Aplana la imagen 28x28 a 784 datos
    tf.keras.layers.Dense(128, activation='relu'), # Capa oculta con 128 neuronas
    tf.keras.layers.Dense(10, activation='softmax') # Capa de salida: 10 dígitos (0-9)
])

# 2. Compilar el modelo (configurar cómo va a aprender)
modelo.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',

```



```

        metrics=['accuracy'])

# 3. Entrenar el modelo (1 sola pasada rápida)
print("\nIniciando entrenamiento rápido (1 época)...")
modelo.fit(x_train, y_train, epochs=1)

# 4. Evaluar el modelo con datos que nunca ha visto (x_test)
print("\n=== Evaluando la Precisión ===")
perdida, precision = modelo.evaluate(x_test, y_test, verbose=2)

# Reemplazamos el punto por la coma para el formato regional español en la consola
precision_porcentaje = f"{precision * 100:.2f}".replace('.', ',')
print(f"\n¡Listo! El modelo tiene una precisión del {precision_porcentaje} % reconociendo números nuevos.")

```

Anatomía de la red (3 capas)

Flatten transforma la imagen 28×28 en un vector de 784 valores. **Dense(128, relu)** es la capa oculta: 128 neuronas con activación ReLU ($\max(0, x)$), que introduce no linealidad. **Dense(10, softmax)** es la capa de salida: 10 neuronas (una por dígito) cuya activación softmax devuelve una distribución de probabilidad que suma 1.

3.4 Script completo consolidado

Este sería el código completo a ejecutar, que une las tres partes anteriores en un único archivo. Se ha actualizado la sintaxis de la capa de entrada para evitar el *UserWarning* que aparece en TensorFlow 2.21 al usar `input_shape` dentro de `Flatten`; en su lugar se declara explícitamente con `tf.keras.Input(shape=(28, 28))`:

Python · [demo_tensorflow_completo.py](#)

```

import tensorflow as tf

print("=== 1. INICIANDO TENSORFLOW ===")
print(f"Versión instalada: {tf.__version__}")

nodo1 = tf.constant(5.0)
nodo2 = tf.constant(6.0)
resultado = tf.multiply(nodo1, nodo2)
print(f"Prueba del motor (Tensores): {nodo1.numpy()} x {nodo2.numpy()} = {resultado.numpy()}\n")

print("=== 2. DESCARGANDO DATASET MNIST ===")
# Cargamos los datos sin el bloque try-except para garantizar que las variables se creen
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

```

```
x_train, x_test = x_train / 255.0, x_test / 255.0
print(f"¡Éxito! {len(x_train)} imágenes cargadas en memoria.\n")

print("=== 3. CONSTRUYENDO Y ENTRENANDO LA RED NEURONAL ===")
# Arquitectura actualizada para evitar el UserWarning en TF 2.21
modelo = tf.keras.models.Sequential([
    tf.keras.Input(shape=(28, 28)),          # Nueva sintaxis para definir la entrada
    tf.keras.layers.Flatten(),               # Aplana la imagen
    tf.keras.layers.Dense(128, activation='relu'), # Capa oculta
    tf.keras.layers.Dense(10, activation='softmax') # Capa de salida (10 dígitos)
])

modelo.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

print("Iniciando entrenamiento rápido (1 época)...")
modelo.fit(x_train, y_train, epochs=1)

print("\n=== 4. EVALUANDO LA PRECISIÓN ===")
perdida, precision = modelo.evaluate(x_test, y_test, verbose=2)

# Formato español para los decimales
precision_porcentaje = f"{precision * 100:.2f}".replace('.', ',')
print(f"\n¡Misión cumplida! Precisión con números nuevos: {precision_porcentaje} %")
```

3.5 Ejecución real en el equipo local

Las dos capturas siguientes muestran la salida real obtenida al ejecutar el script completo en el equipo del alumno. Se observa: (1) la versión instalada de TensorFlow, (2) la multiplicación de tensores como prueba del motor, (3) la carga de las 60 000 imágenes de entrenamiento de MNIST y (4) el resultado final con la precisión obtenida sobre el conjunto de test tras una sola época de entrenamiento.

```

Python 3.12 (64-bit)
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import tensorflow as tf
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1783005834.523704      17224 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to
rors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1783005839.782039      17224 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to
rors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
>>>
>>> print("=== 1. INICIANDO TENSORFLOW ===")
=== 1. INICIANDO TENSORFLOW ===
>>> print(f"Versión instalada: {tf.__version__}")
Versión instalada: 2.21.0
>>>
>>> nodo1 = tf.constant(5.0)
I0000 00:00:1783005841.251378      17224 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in per
s.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX, in other operations, rebuild TensorFlow with the appropriate compiler flags.
>>> nodo2 = tf.constant(6.0)
>>> resultado = tf.multiply(nodo1, nodo2)
>>> print(f"Prueba del motor (Tensores): {nodo1.numpy()} x {nodo2.numpy()} = {resultado.numpy()}\n")
Prueba del motor (Tensores): 5.0 x 6.0 = 30.0
>>>
>>> print("=== 2. DESCARGANDO DATASET MNIST ===")
=== 2. DESCARGANDO DATASET MNIST ===
>>> # Cargamos los datos sin el bloque try-except para garantizar que las variables se creen
>>> mnist = tf.keras.datasets.mnist
>>> (x_train, y_train), (x_test, y_test) = mnist.load_data()
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
+--[1m11490434/11490434+--[0m+--[32m+--[0m+--[37m+--[0m+--[1m1s+--[0m 0us/step
>>> x_train, x_test = x_train / 255.0, x_test / 255.0
>>> print(f"¡Éxito! {len(x_train)} imágenes cargadas en memoria.\n")
¡Éxito! 60000 imágenes cargadas en memoria.

```

Captura 6 — Salida en consola: import, versión, prueba de tensores y carga del dataset MNIST.

```

>>>
>>> print("=== 3. CONSTRUYENDO Y ENTRENANDO LA RED NEURONAL ===")
=== 3. CONSTRUYENDO Y ENTRENANDO LA RED NEURONAL ===
>>> # Arquitectura actualizada para evitar el UserWarning en TF 2.21
>>> modelo = tf.keras.models.Sequential([
...     tf.keras.Input(shape=(28, 28)),           # Nueva sintaxis para definir la entrada
...     tf.keras.layers.Flatten(),                # Aplana la imagen
...     tf.keras.layers.Dense(128, activation='relu'), # Capa oculta
...     tf.keras.layers.Dense(10, activation='softmax') # Capa de salida (10 dígitos)
... ])
>>>
>>> modelo.compile(optimizer='adam',
...                 loss='sparse_categorical_crossentropy',
...                 metrics=['accuracy'])
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/c
WSL2 or the TensorFlow-DirectML plugin.
>>>
>>> print("Iniciando entrenamiento rápido (1 época)...")
Iniciando entrenamiento rápido (1 época)...
>>> modelo.fit(x_train, y_train, epochs=1)
+--[1m1875/1875+--[0m+--[32m+--[0m+--[37m+--[0m+--[1m10s+--[0m 5ms/step - accuracy: 0.9285 - loss: 0.2531
<keras.src.callbacks.history.History object at 0x00000198D3A5F3B0>
>>>
>>> print("\n=== 4. EVALUANDO LA PRECISIÓN ===")

=== 4. EVALUANDO LA PRECISIÓN ===
>>> perdida, precision = modelo.evaluate(x_test, y_test, verbose=2)
313/313 - 1s - 3ms/step - accuracy: 0.9601 - loss: 0.1321
>>>
>>> # Formato español para los decimales
>>> precision_porcentaje = f"{precision * 100:.2f}".replace('.', ',')
>>> print(f"\n¡Misión cumplida! Precisión con números nuevos: {precision_porcentaje} %")

¡Misión cumplida! Precisión con números nuevos: 96,01 %
>>>

```

Captura 7 — Resultado del entrenamiento y la evaluación: precisión final obtenida con una sola época.

4 Glosario de términos clave

Para afianzar los conceptos utilizados durante la presentación, se recogen a continuación las definiciones de los términos técnicos más relevantes del ejercicio. Estas definiciones están alineadas con el glosario oficial de la documentación de TensorFlow y Keras.

Término	Definición
---------	------------

Tensor	Estructura de datos multidimensional (escalar, vector, matriz o array de n dimensiones) con la que trabaja el motor de TensorFlow. Todos los datos que entran y salen de un modelo son tensores.
tf.keras	API de alto nivel integrada en TensorFlow para definir modelos de aprendizaje profundo mediante una sintaxis declarativa. Es la interfaz recomendada para empezar a usar TensorFlow.
Época (epoch)	Una pasada completa del algoritmo de aprendizaje por todo el conjunto de entrenamiento. En el ejemplo se entrena durante una sola época para que la demostración sea rápida.
Neurona	Unidad básica de una red neuronal. Calcula una combinación lineal de sus entradas y le aplica una función de activación no lineal (en el ejemplo, ReLU o softmax).
ReLU	Rectified Linear Unit. Función de activación definida como $f(x) = \max(0, x)$. Es la activación por defecto en capas ocultas por su simplicidad y buen comportamiento en el descenso de gradiente.
Softmax	Función de activación para la capa de salida en clasificación multiclase. Convierte las salidas del modelo en una distribución de probabilidad (valores entre 0 y 1 que suman 1).
Adam	Optimizador adaptativo que combina las ideas de Momentum y RMSProp. Es el optimizador más utilizado por defecto en la mayoría de modelos de Keras.
MNIST	Modified National Institute of Standards and Technology dataset. Conjunto de 70 000 imágenes de dígitos manuscritos (28×28 píxeles) dividido en 60 000 entrenamiento + 10 000 test. Es el «Hola Mundo» del aprendizaje profundo.
CUDA / cuDNN	Librerías de NVIDIA para aceleración de cálculo en GPU. TensorFlow las enlaza automáticamente cuando detecta una GPU compatible, multiplicando el rendimiento.
Wheel (.whl)	Formato de paquete precompilado de Python. pip descarga el wheel correspondiente a tu SO, arquitectura y versión de Python; si no existe, falla con <i>No matching distribution found</i> .

Tabla 2 — Glosario de los términos técnicos utilizados en este ejercicio.

5 Conclusión

TensorFlow es una de las piezas centrales del ecosistema Python para Inteligencia Artificial. Su combinación de una API de alto nivel (*tf.keras*) con un motor de cálculo de bajo nivel escrito en C++ permite construir modelos de aprendizaje automático que se ejecutan con la misma sintaxis en un portátil, en un servidor con GPU o en un dispositivo móvil. La actividad realizada ha permitido verificar empíricamente todo el flujo: preparación del entorno (con downgrade a Python 3.12 para garantizar la compatibilidad con TensorFlow 2.21 y CUDA), instalación mediante *pip*, carga del dataset MNIST y entrenamiento real de una red neuronal secuencial de tres capas que, con una sola época, ya es capaz de reconocer dígitos manuscritos con una precisión notable.

Más allá del ejemplo, lo relevante es que el mismo patrón de código —definir arquitectura, compilar, entrenar y evaluar— se aplica a problemas mucho más complejos: clasificación de imágenes médicas, modelos de lenguaje, detección de fraude o sistemas de recomendación. La curva de aprendizaje inicial se ve compensada con creces por la capacidad productiva de la plataforma: una vez entrenado, un modelo de TensorFlow puede servirse en producción con TensorFlow Serving, exportarse a móvil con TensorFlow Lite o ejecutarse en el navegador con TensorFlow.js. Esa portabilidad es la razón por la que TensorFlow sigue siendo una

herramienta de referencia en la industria del análisis de datos y la inteligencia artificial.

Nota: este análisis tiene finalidad formativa y ha sido elaborado para su exposición en clase. Las versiones mencionadas (Python 3.12, TensorFlow 2.21) corresponden a las disponibles en el momento de realización del ejercicio; se recomienda consultar la documentación oficial para verificar la compatibilidad vigente en el momento de cada nueva instalación.

6 Fuentes y referencias

- › **TensorFlow — documentación oficial:** <https://www.tensorflow.org/> (último acceso: julio 2026)
- › **TensorFlow — guía de instalación:** <https://www.tensorflow.org/install>
- › **tf.keras — Sequential API:** https://www.tensorflow.org/guide/keras/sequential_model
- › **MNIST database — Wikipedia:** https://en.wikipedia.org/wiki/MNIST_database
- › **Python.org — descargas oficiales:** <https://www.python.org/downloads/>
- › **PyPI — paquete tensorflow:** <https://pypi.org/project/tensorflow/>
- › **NVIDIA CUDA Toolkit:** <https://developer.nvidia.com/cuda-toolkit>
- › **Google Brain — DistBelief / TensorFlow paper:** Abadi et al., «TensorFlow: A System for Large-Scale Machine Learning», OSDI 2016.